



Standardization: In-Depth

Step-by-Step Process:

1. Suppose you have a feature Age with values from 18 to 60.
2. Compute the **mean** (μ) and **standard deviation** (σ) of the feature.
3. Apply the formula for each value:

$$z = \frac{x - \mu}{\sigma}$$

4. This transformation results in:
 - o Mean = 0
 - o Standard Deviation = 1

Example:

- Original values: [20, 25, 32, 45, 50]
- Mean (μ) = 34.4, Std Dev (σ) = 12.3
- Transformed values (z-scores): [-1.17, -0.76, -0.20, 0.86, 1.26]



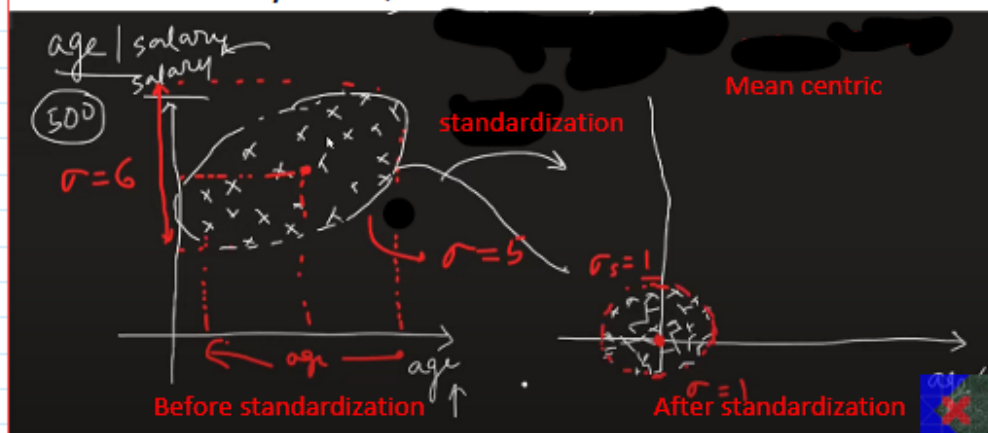
Properties of Standardized Data:

- Mean becomes 0
- Standard deviation becomes 1
- It helps **align scales** of different features, making them **equally influential**



Geometric Intuition

- If you plot data with two features (e.g., Age and Salary):
 - o Unscaled data → **elongated or skewed clusters**
- Scaled data → **symmetric, circular cluster**



- Algorithms like **KNN**, **SVM**, **K-Means**, **PCA** benefit significantly



Summary

Aspect Explanation

What	Transform features to a similar scale
Why	Prevents features with larger scales from dominating
When	Before feeding data into distance-based or gradient-based models
How	Using Standardization or Normalization
Benefit	Improved model performance and convergence

Topic: Data Standardization using Z-score

Objective:

Transform features like Estimated Salary and Age from raw values into a standardized scale so they can be compared and used effectively in machine learning models.

Conceptual Understanding:

1. Initial Visualization:

- When raw salary data is plotted, it appears as scattered points (not normalized).
- The dataset has variance and different scales across features like Age and Salary.

2. Standardization Goal:

Standardization means transforming the data such that:

- Mean = 0
- Standard Deviation = 1

This is achieved by:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x = value
- μ = mean
- σ = standard deviation

3. Two-Step Process:

a. Mean Centering:

- Shift the entire dataset so that the mean becomes zero.
- E.g., move the cloud of points to the center.

b. Scaling:

- Adjust the spread (standard deviation) so that it becomes 1.
- This compresses or expands the data distribution to a standard normal distribution.

4. Why Standardize?

- Makes feature scales comparable.
- Improves performance of algorithms sensitive to feature scaling (e.g., KNN, SVM, Logistic Regression).
- Helps visualize data more meaningfully.
- Outliers can still influence scaling, so be aware.

Practical Example in Python (Scikit-learn)

1. Dataset Info:

- Data from a social network ad dataset.
- Features: Age, EstimatedSalary.
- Target: Whether the person purchased or not (classification problem).

```
[34]: df = pd.read_csv('social_network_ads.csv')
[35]: df.head(10)
[35]:
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15688375	Female	26	43000	0
3	15603348	Female	27	57000	0
4	15804302	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0

Selecting only relevant columns

```
[37]: df = df.iloc[:,2:]
[37]:
```

	Age	EstimatedSalary	Purchased
0	19	19000	0
1	35	20000	0
2	26	43000	0
3	27	57000	0
4	19	76000	0
...	...	...	...
395	40	41000	1
396	51	23800	1
397	50	20000	1
398	36	33000	0
399	49	56000	1

400 rows × 3 columns

2. Preprocessing Steps:

a. Train-Test Split (Best Practice):

- Always split data before standardizing:

```
train_test_split

[95]: from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(
      df.drop("Purchased",axis=1), #Features (x) : all columns except 'Purchased'
      df["Purchased"],           #Target (y) : the Purchased column
      test_size = 0.3,           # 30% of data for testing, 70% for training
      random_state = 0)         # Ensures reproducibility of the split

[96]: x_train.shape, x_test.shape
[96]: ((280, 2), (120, 2))
```

b. Import and Apply StandardScaler:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Learn and transform on training data
X_test_scaled = scaler.transform(X_test)      # Only transform test data
```

Standard Scaler

```
[72]: from sklearn.preprocessing import StandardScaler
      scaler = StandardScaler()
      #fitting the training data
      scaler.fit(x_train)
      #transforming the data and test data
      x_train_scaled = scaler.transform(x_train)
      x_test_scaled = scaler.transform(x_test)
```

```
[73]: scaler.mean_
```

```
[73]: array([3.78642857e+01, 6.98071429e+04])
```

```
[74]: x_train_scaled
```

```
[74]: array([[ -1.1631724, -1.5849703 ],
        [  2.17018137,  0.93090672],
        [  0.0133054,  1.22017719],
        [  0.20938504,  1.07558195],
        [  0.40546467, -0.48604654],
        [ -0.28081405, -0.31253226],
        [  0.99370357, -0.8330751 ],
        [  0.0000000,  0.0000000 ]])
```

✓ Always fit only on training data and transform both train and test — this avoids data leakage.

3. What happens internally?

- During `.fit()`, the scaler calculates:
 - Mean and standard deviation for each feature.
- During `.transform()`, it applies:

$$z = \frac{x - \mu}{\sigma}$$

4. Working with DataFrames:

- If you pass a DataFrame, the scaler returns a NumPy array, not a DataFrame.
- You need to convert it back:

```
[75]: # since StandardScaler returns array so we will convert it to dataframes first then use it
      x_train_scaled = pd.DataFrame(x_train_scaled, columns = ['Age', 'EstimatedSalary'])
```

```
[76]: x_train.describe()
```

```
[76]:
```

	Age	EstimatedSalary
count	280.000000	280.000000
mean	37.864286	69807.142857
std	10.218201	34641.201654
min	18.000000	15000.000000
25%	30.000000	43000.000000
50%	37.000000	70500.000000
75%	46.000000	88000.000000
max	60.000000	150000.000000

```
import pandas as pd
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)
```

5. Validation:

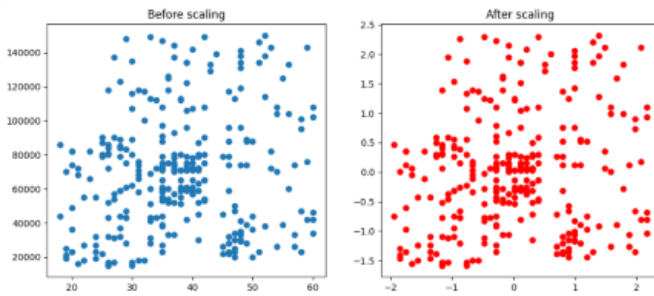
a. Use `.describe()` to compare before and after:

- Original data: Mean $\neq 0$, Std $\neq 1$
- After scaling: Mean ≈ 0 , Std ≈ 1

b. Verify using plots:

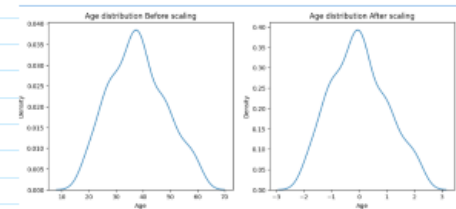
visualising effect of scaling on data

```
[84]: fig, (ax1,ax2) = plt.subplots(ncols = 2, figsize = (12,5))
ax1.scatter(x_train['Age'],x_train['EstimatedSalary'])
ax1.set_title("Before scaling")
ax2.scatter(x_train_scaled['Age'],x_train_scaled['EstimatedSalary'],color='red')
ax2.set_title("After scaling")
plt.show()
```

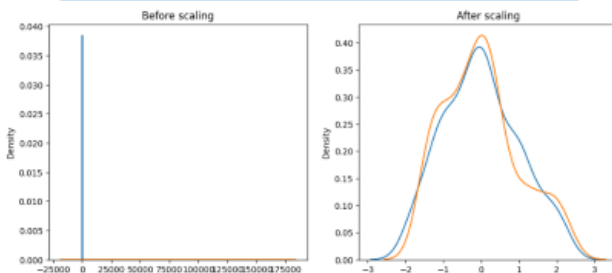


This scatter plot should show a more centralized and compressed distribution.

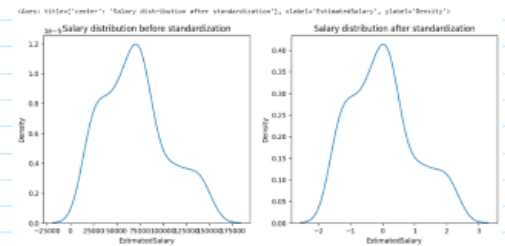
```
[90]: fig2, (ax1,ax2) = plt.subplots(ncols = 2, figsize = (12,5))
ax1.set_title("Age distribution Before scaling")
sns.kdeplot(x_train['Age'],ax = ax1)
ax2.set_title("Age distribution After scaling")
sns.kdeplot(x_train_scaled['Age'],ax = ax2)
plt.show()
```



```
[88]: fig2, (ax1,ax2) = plt.subplots(ncols = 2, figsize = (12,5))
ax1.set_title("Before scaling")
sns.kdeplot(x_train['Age'],ax = ax1)
sns.kdeplot(x_train['EstimatedSalary'],ax = ax1)
ax2.set_title("After scaling")
sns.kdeplot(x_train_scaled['Age'],ax = ax2)
sns.kdeplot(x_train_scaled['EstimatedSalary'],ax = ax2)
plt.show()
```



```
[93]: fig, (ax1,ax2) = plt.subplots(ncols = 2, figsize = (12,5))
ax1.set_title('Salary distribution before standardization')
sns.kdeplot(x_train['EstimatedSalary'],ax = ax1)
ax2.set_title('Salary distribution after standardization')
sns.kdeplot(x_train_scaled['EstimatedSalary'],ax = ax2)
```



6. Effect of Outliers:

- If there are **extreme values**, standardization still includes them in the mean/SD calculation.
- Z-score standardization does not eliminate outliers — it just compresses the scale.

7. Important Takeaways:

- Standardization is useful and often necessary before many ML models.
- Always remember the **sequence**:
 - a. Train-Test Split
 - b. Fit only on Train
 - c. Transform both Train and Test
- Beware of outliers and understand their impact.